

Indicore Originals: Implementation Plan

1.1 Project Summary

Indicore Originals is a modern, mobile-first e-commerce platform selling curated products via Instagram-driven traffic. The MVP supports:

- **Cash on Delivery (COD)** as the sole payment method.
- Single-step checkout with integrated lead collection.
- Product variants, image management, moderated reviews (registered customers only).
- Guest order tracking (Order ID + Email).
- Manual COD verification.
- Homepage content management, FAQs, and analytics.

Key Differentiators:

- Single login page with role-based redirect (admin/customer).
- Guest checkout with automatic order linking upon registration.
- Purchase-validated reviews (only delivered orders can review).
- Manual COD verification workflow (no OTP/SMS/email dependency).
- Fixed 7-section homepage structure (content-managed, not structural).
- All customer communication handled manually by admin via phone or WhatsApp.

1.2 Business Goals

Goal	Success Metric	Target (3 months)
Convert Instagram traffic	Conversion rate	2.5%
Build customer database	Consented emails collected	5,000
Maintain trust via manual verification	COD success rate	85%
Generate repeat purchases	Repeat purchase rate	20%
Collect product feedback	Reviews per order	0.3

1.3 Technical Goals

Goal	Success Metric	Target
Mobile-first performance	Lighthouse mobile score	> 90
Fast page loads	LCP	< 2.5s on 3G

Goal	Success Metric	Target
Secure authentication	No breaches	Zero incidents
Zero-cost MVP	Monthly hosting	\$0 (free tier)
Maintainable codebase	Developer onboarding	< 2 days

1.4 MVP Scope (Version 1)

In Scope:

- COD as sole payment method.
- Single-step checkout with optional consent checkbox (marketing opt-in).
- Product variants (size, color, material, custom).
- Product images (primary + up to 10 gallery, WebP conversion).
- Guest checkout + optional registration.
- Order tracking via Order ID + Email (with State in address).
- Guest order history via Email only.
- Purchase-validated reviews (registered customers only).
- Review moderation (approve/reject/hide/delete).
- Fixed 7-section homepage (content-managed), including Customer Reviews settings.
- FAQ management (create/edit/delete/reorder).
- Manual COD verification workflow.
- Analytics dashboard with date range filtering.
- Admin product/category/tag management.
- Multi-tag product filtering.
- Database-based rate limiting (login + order tracking).

Explicitly Out of Scope (V1):

- Email notifications of any kind (order confirmation, status updates, admin alerts) — Post-V1.
- Automated admin order notifications — Admin monitors dashboard manually.
- SMS notifications / OTP verification.
- Online payment gateways.
- Multiple saved shipping addresses per customer — Post-V1 (single address only in V1).
- Multi-vendor marketplace, affiliate program, mobile apps, multi-language/currency, AI recommendations, live chat, 2FA / IP whitelisting, gamification / loyalty features, inventory forecasting / low-stock alerts.
- Admin idle session timeout (admin manages logout manually).

1.5 Document Conflict Resolution

All conflicts between the PRD and TRD have been resolved below. This implementation plan supersedes both documents where conflicts exist.

#	Conflict / Gap	PRD Position	TRD Position	Resolution Applied
1	Email notifications	"Email is the only automated channel; order confirmation email sent"	"No automated email/SMS notifications — non-goal"	Follow TRD for V1. No email service, no email triggers anywhere. All customer communication is manual (phone/WhatsApp). Post-V1 enhancement.
2	Rate limiting approach	"Implement via Upstash Redis middleware"	"Database-based rate limiting (no Redis)"	Follow TRD. Use <code>customers.login_attempts / locked_until</code> for login; <code>order_track_attempts</code> table for order tracking. No Redis/Upstash dependency.
3	Multiple shipping addresses	Registered customers can save multiple addresses	<code>customers</code> table has single address column; no addresses table exists	Remove from V1. Single shipping address storage per customer. Feature moved to Post-V1 Out of Scope.
4	<code>password_hash</code> in Customer Model	PRD data model lists <code>password_hash</code> field	Supabase Auth manages passwords in <code>auth.users</code>	Remove <code>password_hash</code> from customers table. Supabase Auth handles password storage. No <code>password_hash</code> column anywhere in the customer table.
5	30-minute admin idle timeout	"Session timeout: 30 min admin"	Same, but not implementable natively in Supabase	Remove requirement. Supabase does not support idle timeouts natively; logout is manual. Customer session remains 7 days.
6	<code>homepage_content</code> schema missing Customer Reviews settings	PRD defines <code>customer_reviews</code> : <code>max_reviews_to_show</code> (integer)	<code>homepage_content</code> table has no such column	Add <code>customer_reviews</code> jsonb column to <code>homepage_content</code> (e.g., <code>{"max_reviews_to_show": 6}</code>). Admin editor includes this field.
7	Consent field validation	"Consent checkbox (explicit, not pre-ticked)" for marketing	<code>POST /api/orders</code> lists "all fields present" implying consent required	Consent is optional. It is a marketing opt-in, not a prerequisite for placing an order. consent defaults to false. Orders are accepted with or without consent.
8	State field missing from shipping address	Form collects: street, city, pincode (no State)	Address stored as plain text; no State field	Add State to all address structures. Checkout API payloads, Zod validation, and <code>orders.shipping_address</code> JSON all include state.
9	Product tag filtering (singular)	Products filterable by tags (plural)	API param <code>tag-id</code> (singular)	Support multiple tags: <code>tags=id1,id2,id3</code> (comma-separated). Frontend uses multi-select.
10	Cart not cleared after checkout	Not specified	Cart uses <code>localStorage</code> ; no	Clear <code>localStorage</code> cart immediately after successful order placement, before redirecting to order confirmation page.

#	Conflict / Gap	PRD Position	TRD Position	Resolution Applied
			clear after success	
11 (Gap)	Admin new-order notifications	No mechanism defined in either document	No mechanism defined	No automated notification in V1. Admin manually monitors the Orders dashboard. No push, email, or SMS alert on new order. Explicitly in Out of Scope.

Action Items from Conflict Resolution:

- PRD Section "Communication" must state: *"No automated email or SMS in V1. Customer communication is manual (admin phone/WhatsApp). Order status available via Track Order page."*
- PRD Customer Model must remove password_hash field.
- PRD Customer Features must remove "Save multiple shipping addresses" from V1.
- All address fields in PRD, TRD, and this plan now include: street, city, state, pincode.

2. Development Approach & Strategy

2.1 Recommended Development Approach

Iterative, Feature-First Development with Parallel Tracks.

Track	Focus	Timeline
Track A	Backend & Database	-
Track B	Frontend Storefront	-
Track C	Admin Dashboard	-
Track D	QA & Testing	-

Development Philosophy:

- Build vertical slices (complete features from DB to UI).
- Deploy to staging after each phase.
- No feature branches longer than 3 days.
- Daily integration to develop branch.

2.2 Feature Delivery Strategy

Priority	Feature	Dependency	Risk
P0	Database schema + RLS	None	High

Priority	Feature	Dependency	Risk
P0	Authentication + roles	Database	High
P0	Product management (admin)	Database	Medium
P0	Storefront product display	Product API	Medium
P0	Cart + checkout (with State)	Product + Customer	High
P0	Order creation + inventory	Cart + Product	High
P1	Guest order tracking	Order	Low
P1	Admin order management	Order	Medium
P1	Customer registration	Customer	Low
P1	Wishlist	Customer	Low
P2	Reviews (customer + admin)	Order	Medium
P2	Homepage editor (with customer_reviews config)	CMS	Low
P2	FAQ management	CMS	Low
P2	Analytics	Order	Low

2.3 Risk Reduction Strategy

Risk	Mitigation	Timeline
Supabase RLS complexity	Build + test RLS policies in Phase 0 (prototype)	-
Variant inventory concurrency	Use database row-level locks in order creation	-
Rate limiting edge cases	Test with k6 load testing	-
Guest order linking race condition	Use database transaction with unique constraint	-
Image upload performance	Implement client-side resizing before upload	-

3. Phase-by-Phase Implementation Plan

Phase 0: Foundation & Infrastructure Setup

Objective: Establish development environment, Supabase project, and foundational code structure.

Activity	Details
Supabase project creation	New project (indicore-originals-prod) + (indicore-originals-staging)
GitHub repository setup	Branch protection rules, main/develop/staging branches
Vercel project connection	Connect GitHub, configure staging/production environments
Local development environment	Docker Compose with local Supabase, Node.js 20+
Shared component library	shadcn/ui setup, Tailwind configuration

Deliverables:

- 1. Supabase projects created (staging + production).
- 2. GitHub repo with branch protection.
- 3. Vercel deployments auto-deploying from staging and main.
- 4. Local development environment documented in README.

Phase 1: Database & Authentication

Objective: Complete database schema with RLS policies, authentication system with role-based redirect.

Database Work

Tables to Create (in order):

```
sql
-- Run via Supabase migration
-- 1. customers (extends auth.users - no password_hash column)
-- 2. categories
-- 3. tags
-- 4. products
-- 5. product_variants
-- 6. product_tags
-- 7. orders
-- 8. order_items
-- 9. reviews
-- 10. wishlist
-- 11. faq
-- 12. homepage_content (includes customer_reviews jsonb column)
-- 13. settings
```

-- 14. order_track_attempts

Key Implementation Details:

Requirement	Implementation
is_out_of_stock	BEFORE INSERT OR UPDATE trigger; set NEW.is_out_of_stock := (NEW.stock_quantity <= 0) — DO NOT update same table
Phone non-unique	No unique constraint on customers.phone
Email unique	Unique constraint on customers.email
Guest customers	is_registered = false; no password_hash column — Supabase Auth owns all passwords
Admin idle timeout	Not implemented. Admin logs out manually. Customer session: 7 days.
Shipping address	Stored as JSON with fields: street, city, state, pincode

homepage_content table - includes Customer Reviews settings:

sql

```
CREATE TABLE homepage_content (  
  id bigserial PRIMARY KEY,  
  hero_banner jsonb,  
  featured_product_ids bigint[],  
  categories_section jsonb,  
  why_choose_us jsonb,  
  customer_reviews jsonb DEFAULT '{"max_reviews_to_show": 6}',  
  footer jsonb,  
  updated_at timestamp DEFAULT now()  
);
```

-- Singleton: only one row allowed

RLS Policies (Mandatory):

- Enable RLS on all tables.
- **Customers:** users read/update own, admins all.

sql

```
CREATE POLICY "Users read own customer row" ON customers FOR SELECT USING (auth.uid() = id);  
CREATE POLICY "Users update own customer row" ON customers FOR UPDATE USING (auth.uid() = id);  
CREATE POLICY "Allow guest customer creation" ON customers FOR INSERT WITH CHECK (true);  
CREATE POLICY "Admins all customers" ON customers FOR ALL USING ( (SELECT role FROM customers WHERE id = auth.uid()) = 'admin' );
```

- **Products & Variants:** public read, admin write.
- **Orders:** customers read own; allow INSERT for guest checkout; admin all.

- **Order Items:** allow INSERT with orders.
- **Reviews:** public read approved; customers create/update own; admin all.
- **FAQs, homepage_content, settings:** public read; admin write.
- **order_track_attempts:** allow insert from public (rate limiting); admin read.

Backend Work (Authentication Endpoints)

Endpoint	Method	Purpose
/api/auth/login	POST	Email/password login, DB rate limiting, role redirect
/api/auth/signup	POST	Customer registration
/api/auth/logout	POST	Destroy session
/api/auth/session	GET	Get current session + role

Rate Limiting Logic (Login - Database-based, no Redis):

typescript

// Database-based login rate limiting - no external dependencies

```
async function checkLoginRateLimit(email: string): Promise<{ allowed: boolean; remainingMinutes: number }> {
  const customer = await db.customers.findUnique({ where: { email } });
  if (customer?.locked_until && customer.locked_until > new Date()) {
    const remainingMinutes = Math.ceil((customer.locked_until.getTime() - Date.now()) / 60000);
    return { allowed: false, remainingMinutes };
  }
  if (customer && customer.login_attempts >= 5) {
    await db.customers.update({
      where: { email },
      data: { locked_until: new Date(Date.now() + 15 * 60 * 1000) }
    });
    return { allowed: false, remainingMinutes: 15 };
  }
  return { allowed: true, remainingMinutes: 0 };
}
```

Middleware Implementation:

typescript

```
import { createMiddlewareClient } from '@supabase/auth-helpers-nextjs';
import { NextResponse } from 'next/server';
```

```
export async function middleware(req) {
```

```

const res = NextResponse.next();

const supabase = createMiddlewareClient( { req, res } );

const { data: { session } } = await supabase.auth.getSession();

// Admin route protection
if (req.nextUrl.pathname.startsWith('/admin')) {
  if (!session) {
    return NextResponse.redirect(new URL('/login', req.url));
  }

  const { data: customer } = await supabase
    .from('customers')
    .select('role')
    .eq('id', session.user.id)
    .single();

  if (customer?.role !== 'admin') {
    return NextResponse.redirect(new URL('/login', req.url));
  }
}

if (req.nextUrl.pathname.startsWith('/customer')) {
  if (!session) {
    return NextResponse.redirect(new URL('/login', req.url));
  }
}

return res;
}

```

```

export const config = {
  matcher: ['/admin/:path*', '/customer/:path*']
};

```

Note: Admin session timeout is NOT implemented. Supabase does not support idle timeouts natively. Admin must log out manually. Customer sessions persist for 7 days.

Testing Requirements:

- Admin login redirects to /admin/dashboard.
- Customer login redirects to /customer/dashboard.
- 5 failed login attempts → 15-minute lockout (database-based).
- Admin route inaccessible to customers.
- Guest customer insertion succeeds via RLS INSERT policy.

Deliverables:

- Working authentication with role-based redirect.
- Database schema deployed to staging.
- RLS policies tested.
- Login page with rate-limiting error messages.

Phase 2: Product Management (Admin) (Days 11-17)

Objective: Complete admin product CRUD with variants, images, categories, tags, and SEO fields.

Database Work

Table	Operations
products	INSERT, UPDATE, DELETE (admin only via RLS)
product_variants	INSERT, UPDATE, DELETE (cascade with product)
categories	CRUD
tags	CRUD
product_tags	Junction table management

Slug Uniqueness:

sql

```
ALTER TABLE products ADD CONSTRAINT products_slug_unique UNIQUE (slug);
```

```
CREATE OR REPLACE FUNCTION generate_unique_slug(base_slug TEXT)
```

```
RETURNS TEXT AS $$
```

```
DECLARE
```

```
    new_slug TEXT;
```

```
    counter INT := 0;
```

```
BEGIN
```

```
    new_slug := base_slug;
```

```
    WHILE EXISTS (SELECT 1 FROM products WHERE slug = new_slug) LOOP
```

```
        counter := counter + 1;
```

```
        new_slug := base_slug || '-' || counter;
```

```
    END LOOP;
```

```
    RETURN new_slug;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Backend Work (Admin Product Endpoints)

Endpoint	Method	Purpose
/api/admin/products	GET	List products with filters
/api/admin/products	POST	Create product with variants
/api/admin/products/:id	GET/PUT/DELETE	Single product operations
/api/admin/products/:id/variants	POST	Add variant
/api/admin/products/:id/variants/:vid	PUT/DELETE	Variant operations
/api/admin/categories	CRUD	Category management
/api/admin/tags	CRUD	Tag management
/api/admin/upload/image	POST	Upload to Supabase Storage, convert to WebP

Image Upload Requirements:

typescript

// Must implement:

// - Accept: JPG, JPEG, PNG, WebP

// - Max size: 5MB

// - Resize: max width 1200px (gallery), 600px (thumbnails)

// - Convert to WebP (quality 80)

// - Upload to Supabase Storage bucket 'product-images'

// - Return public URL

// - Validate MIME type server-side (never trust client)

Variant Validation:

- SKU must be unique across all variants.
- Each variant has stock_quantity (default 0).
- is_out_of_stock updated automatically via trigger.

Frontend Work (Admin)

Screen	Components
/admin/products	Data table with search, filter by category, pagination
/admin/products/new	Form: name, description, price, category, SEO fields, slug editor

Screen	Components
/admin/products/[id]/edit	Same as new + variant manager, image gallery
/admin/categories	List + create/edit modal
/admin/tags	Simple list + add/delete

Testing Requirements:

- Create product with 3 variants → variants saved correctly.
- Edit product → slug uniqueness enforced.
- Upload image → converted to WebP, URL returned.
- Delete product → variants cascade deleted.
- Out-of-stock variant → `is_out_of_stock` = true automatically.
- SKU uniqueness violation → error message.

Deliverables:

- Fully functional admin product management.
- Image upload with WebP conversion.
- Variant management working.
- Category and tag CRUD.

Phase 3: Storefront & Product Display

Objective: Customer-facing product browsing, multi-tag filtering, search, and cart.

Backend Work (Public Endpoints)

Endpoint	Method	Purpose
/api/products	GET	Public product listing with filters
/api/products/:slug	GET	Public product detail
/api/categories	GET	Public categories
/api/tags	GET	Public tags
/api/search	GET	Search products

Product Listing Filters (multi-tag support):

typescript

// Query parameters supported:

```
// ?category=slug

// ?tags=id1,id2,id3 <-- comma-separated; multiple tags supported (Resolution #9)

// ?inStock=true

// ?minPrice=100&maxPrice=5000

// ?search=black+tshirt

// ?sort=price_asc|price_desc|newest|popularity

// ?page=1&limit=20


// Tag parsing on API:

const tagIds = req.query.tags ? String(req.query.tags).split(',').map(Number).filter(Boolean) : [];

// Apply to query:

if (tagIds.length > 0) {

  query = query.filter('product_tags.tag_id', 'in', `(${tagIds.join(',')})`);

}
```

Product Detail Response must include:

- All variants with stock status.
- Gallery images (array of URLs).
- Approved reviews (filtered by status = 'approved').
- SEO metadata (title, description).
- Average rating (aggregate from approved reviews).

Frontend Work (Customer)

Screen	Components	Priority
Homepage	7 fixed sections (fetch from homepage_content)	P0
Product Listing	Grid with filters sidebar (mobile: drawer)	P0
Product Detail	Variant selector, gallery carousel, add to cart	P0
Cart	List items, quantity update, remove, total	P0
Search Results	Same as product listing	P1

Homepage Fixed Sections (DO NOT DEVIATE - 7 sections exactly):

1. Hero Banner (image, text, CTA)
2. Featured Products (selected by admin)
3. Categories Section (display categories)
4. Why Choose Us (USPs with icons)
5. Customer Reviews (latest approved; count controlled by customer_reviews.max_reviews_to_show from homepage_content)

6. FAQ Section (from FAQ table, ordered)
7. Footer (contact, social, copyright)

Multi-Tag Filter UI:

tsx

// FilterSidebar - tags use multi-select checkboxes

```
const [selectedTags, setSelectedTags] = useState<number[]>([]);
```

```
const toggleTag = (tagId: number) => {
```

```
  setSelectedTags(prev =>
```

```
    prev.includes(tagId)
```

```
      ? prev.filter(id => id !== tagId)
```

```
      : [...prev, tagId]
```

```
  );
```

```
};
```

// Build URL param when selectedTags changes

```
const tagParam = selectedTags.length > 0 ? `tags=${selectedTags.join(',')}` : '';
```

Cart State Management:

typescript

// React Context + localStorage

```
interface CartItem {
```

```
  variantId: number;
```

```
  productId: number;
```

```
  productName: string;
```

```
  variantDescription: string; // e.g. "Size: M, Color: Black"
```

```
  price: number;
```

```
  quantity: number;
```

```
  imageUrl: string;
```

```
  stockAvailable: number;
```

```
}
```

// Cart context must expose:

```
function clearCart(): void {
```

```
  localStorage.removeItem('indicore_cart');
```

```
  setItems([]);
```

```
}
```

// Called immediately after successful order placement (see Phase 4)

Testing Requirements:

- Product listing multi-tag filter works (selecting 2 tags shows products with either tag).
- Search returns relevant results.

- Product detail shows correct variants with stock status.
- Cannot add out-of-stock variant to cart.
- Cart persists after page refresh.
- Mobile responsive (320px - 1440px).
- Lighthouse mobile score > 90.

Deliverables:

- Fully functional customer storefront.
- Multi-tag product filtering.
- Cart system working.

Phase 4: Checkout & Order Creation

Objective: Complete single-step checkout with State field, optional consent, COD validation, order creation, inventory deduction, cart clearing, and order confirmation.

Order Creation Transaction (with Row-Level Locks):

sql

-- 1. Lock variants for update (row-level lock)

SELECT * FROM product_variants WHERE id IN (variant_ids) FOR UPDATE;

-- 2. Verify stock sufficient

-- 3. Deduct inventory

UPDATE product_variants

SET stock_quantity = stock_quantity - quantities

WHERE id IN (variant_ids);

-- 4. Create order (shipping_address stored as JSON with state field)

INSERT INTO orders (

customer_id, order_number, status, cart_total, shipping_address, consent_given

) VALUES (

..., ..., 'pending', ..., '{"street": "...", "city": "...", "state": "...", "zipcode": "...}', ...

);

-- 5. Create order items

INSERT INTO order_items (order_id, variant_id, quantity, unit_price, variant_snapshot) VALUES (...);

COMMIT;

orders.shipping_address **JSON structure:**

json

{

"street": "123 Main St",

```
"city": "Mumbai",  
"state": "Maharashtra",  
"pincode": "400001"  
}
```

Order Number Generation Function:

```
sql  
  
CREATE OR REPLACE FUNCTION generate_order_number()  
RETURNS TEXT AS $$  
  
DECLARE  
  
    timestamp_part TEXT;  
  
    random_part TEXT;  
  
BEGIN  
  
    timestamp_part := EXTRACT(EPOCH FROM NOW())::TEXT;  
  
    random_part := UPPER(SUBSTRING(MD5(RANDOM())::TEXT FROM 1 FOR 6));  
  
    RETURN 'ORD-' || timestamp_part || '-' || random_part;  
  
END;  
  
$$ LANGUAGE plpgsql;
```

Backend Work (Checkout Endpoints)

Endpoint	Method	Purpose
/api/orders	POST	Create order (guest or customer)
/api/orders/validate	POST	Validate cart before checkout (COD min/max)
/api/checkout/settings	GET	Get COD min/max values

POST /api/orders - Request Body:

```
json  
  
{  
  
    "email": "user@example.com",  
  
    "name": "Rahul Sharma",  
  
    "phone": "9876543210",  
  
    "address": {  
  
        "street": "123 Main Street, Sector 5",  
  
        "city": "Gurgaon",  
  
        "state": "Haryana",  
  
        "pincode": "122001"  
  
    },  
  
    "cart": [  

```

```

    { "variant_id": 123, "quantity": 2 }
  ],
  "consent": false
}

```

Order Creation Logic:

typescript

// POST /api/orders

```

async function createOrder(req, res) {
  // 1. Validate all fields (Zod schema - consent is optional)

  const { email, name, phone, address, cart, consent } = req.body;

  // 2. Get COD settings

  const { cod_min, cod_max } = await getSettings();

  const cartTotal = calculateCartTotal(cart);

  // 3. Validate COD range

  if (cartTotal < cod_min || cartTotal > cod_max) {
    return res.status(400).json({ error: `COD available for orders between ${cod_min} and ${cod_max}` });
  }

  // 4. Find or create customer (email unique, phone non-unique)

  const customer = await db.customers.upsert({
    where: { email },
    update: { phone, name, address: JSON.stringify(address) },
    create: {
      email, phone, name, address: JSON.stringify(address),
      is_registered: false, role: 'customer', email_consent_given: consent ?? false
    }
  });

  // 5. Create order with transaction

  const order = await createOrderTransaction(cart, customer, cartTotal, address);

  // 6. Return order number for confirmation page

  // NOTE: No email notification is sent. Admin monitors dashboard manually.

  return res.json({ success: true, orderNumber: order.order_number });
}

```

Frontend Work (Checkout Page)

Form Fields (with State):

tsx

// Single-step checkout form

```

<form onSubmit={handleSubmit}>
  <input name="fullName" placeholder="Full Name" required />
  <input name="phone" placeholder="Phone Number" required />

```

```

<input name="street" placeholder="Street Address" required />
<input name="city" placeholder="City" required />
<input name="state" placeholder="State" required />
<input name="pincode" placeholder="Pincode" required />
<input name="email" placeholder="Email (for order updates)" required />
{ /* Resolution #7: consent is optional - NOT pre-ticked */ }
<label>
  <input type="checkbox" name="consent" defaultChecked={false} />
  I agree to receive order updates and marketing communications from Indicore
</label>
<button type="submit" disabled={!placeOrderEnabled}>Place Order</button>
</form>

```

COD Min/Max Enforcement:

```

tsx
useEffect(() => {
  fetch('/api/checkout/settings')
    .then(res => res.json())
    .then(settings => {
      if (cartTotal < settings.cod_min) {
        setError(`COD available for orders above ₹${settings.cod_min}`);
        setPlaceOrderEnabled(false);
      } else if (cartTotal > settings.cod_max) {
        setError(`COD available for orders up to ₹${settings.cod_max}`);
        setPlaceOrderEnabled(false);
      } else {
        setError(null);
        setPlaceOrderEnabled(true);
      }
    });
}, [cartTotal]);

```

Cart Clearing After Successful Order (Resolution #10):

```

typescript
// Immediately after receiving successful response from POST /api/orders
const handleOrderSubmit = async (formData) => {
  try {
    const result = await fetch('/api/orders', {
      method: 'POST',
      body: JSON.stringify(buildOrderPayload(formData)),
    }).then(r => r.json());
  }

```

```

if (result.success) {
    clearCart(); // Resolution #10: clear localStorage cart BEFORE redirect
    router.push(`/order/confirmation/${result.orderNumber}`);
}
} catch (err) {
    setError('Order placement failed. Please try again.');
```

Consent Checkbox Requirements:

- NOT pre-ticked (default unchecked).
- Label: "I agree to receive order updates and marketing communications".
- Optional - order placement proceeds regardless of whether checkbox is checked.
- consent value (true/false) stored in orders.consent_given as a snapshot.
- If true, also update customers.email_consent_given = true.
- No email notifications are triggered at any point in this flow.

Testing Requirements:

- Checkout form validates all required fields (name, phone, street, city, state, pincode, email).
- State field is required and validated.
- COD min/max blocks out-of-range orders.
- Place Order disabled with error message when out of range.
- Order placed with consent=false succeeds (consent is optional).
- Order placed with consent=true also succeeds; email_consent_given updated.
- Order created with status = pending.
- Inventory deducted correctly.
- Guest customer created (is_registered = false), no password_hash column.
- Same email reuses customer record (updates phone/address).
- Cart is cleared from localStorage immediately after order success.
- Order confirmation page displays order number.
- Concurrent orders (same variant) handled correctly via row-level locks.
- No email or notification sent after order placement.

Deliverables:

- Working checkout flow with State field.
- Optional consent checkbox.
- Order creation with inventory deduction.

- Cart clearing before redirect to confirmation.
- Order confirmation page.

Phase 5: Order Management & Tracking

Objective: Admin order management, guest order tracking, order history, and guest-to-registered account linking.

Admin Order Monitoring (Resolution #11): There are no automated notifications when a new order is placed. The admin must manually check the Orders dashboard to see incoming orders. The admin dashboard should display a count of pending orders prominently to facilitate manual monitoring.

Database Work

Order Status Flow: pending → confirmed → shipped → delivered (cancelled/return requested can also occur).

Cleanup for order track attempts:

- Option 1: pg_cron (if available on Supabase plan)
- Option 2: Vercel cron job calling /api/admin/cleanup-tracking

```
sql
CREATE OR REPLACE FUNCTION cleanup_order_track_attempts()
RETURNS void AS $$
BEGIN
    DELETE FROM order_track_attempts WHERE expires_at < NOW();
END;
$$ LANGUAGE plpgsql;
```

Backend Work (Order Endpoints)

Endpoint	Method	Purpose
/api/orders/track	GET	Guest tracking (order number + email)
/api/orders/guest-history	GET	List orders by email
/api/admin/orders	GET	Admin order list with filters
/api/admin/orders/:id	GET	Order detail
/api/admin/orders/:id/status	PATCH	Update status
/api/admin/orders/:id/verify-cod	POST	Mark as confirmed (manual verification)
/api/admin/orders/:id/return	POST	Process return request

Rate Limiting - Order Tracking (Database-based, no Redis):

```
typescript
const TRACKING_LIMIT = 10;
const TRACKING_WINDOW_MS = 15 * 60 * 1000;
```

```
async function checkTrackingRateLimit(ip: string): Promise<{ allowed: boolean }> {
  const windowStart = new Date(Date.now() - TRACKING_WINDOW_MS);
  const attempts = await db.order_track_attempts.count({
    where: {
      ip_address: ip,
      attempt_time: { gte: windowStart }
    }
  });
  if (attempts >= TRACKING_LIMIT) {
    return { allowed: false };
  }
  await db.order_track_attempts.create({
    data: {
      ip_address: ip,
      attempt_time: new Date(),
      expires_at: new Date(Date.now() + 24 * 60 * 60 * 1000)
    }
  });
  return { allowed: true };
}
```

Guest-to-Registered Account Linking:

```
typescript
// POST /api/auth/link-orders (called during registration)
async function linkGuestOrders(email: string, customerId: string) {
  const guestCustomer = await db.customers.findUnique({ where: { email } });
  if (!guestCustomer || guestCustomer.is_registered) return;

  await db.orders.updateMany({
    where: { customer_id: guestCustomer.id },
    data: { customer_id: customerId }
  });
}
```

Customer Screens

Screen	Access	Features
/track-order	Guest (main nav)	Form: Order Number + Email → display status + address (with State)

Screen	Access	Features
/order-history	Guest (email only)	Enter email → list all orders
/customer/orders	Registered	Full order history
/customer/orders/[id]	Registered	Single order detail with State in address

Admin Screens

Screen	Features
/admin/orders	Table with status filters, search by order number, date range; pending count badge prominently displayed for manual monitoring
/admin/orders/[id]	Order detail with full address (including State), status update buttons, return management

COD Verification Workflow (Admin - Manual, No Notifications):

1. Admin opens /admin/orders - sees list sorted by newest; pending orders highlighted.
2. Admin sees pending order > clicks to open detail.
3. Admin calls customer or sends WhatsApp message (phone number displayed).
4. Customer confirms order.
5. Admin clicks "Mark as Confirmed" button.
6. Order status changes to "confirmed".
7. (Optional) Admin adds internal note about verification call.

Testing Requirements:

- Guest can track order with Order Number + Email.
- Rate limiting blocks after 10 attempts per IP (database-based, no Redis).
- Guest can view all orders by entering email.
- Admin can update status: pending → confirmed → shipped → delivered.
- Admin can cancel order.
- Guest registers with same email → prompt to link orders.
- Linked orders appear in registered customer dashboard.
- Return request flow works.
- No email sent at any status transition.
- Admin pending order count displays correctly on orders table.

Deliverables:

- Guest order tracking working.
- Admin order management with manual monitoring support (pending count badge).
- Account linking functional.

Phase 6: Customer Features & Reviews (Days 41-50)

Objective: Customer registration, dashboard, wishlist, and purchase-validated review system with admin moderation.

Note on Addresses: V1 supports a single shipping address per customer. The customers.address column stores JSON ({"street","city","state","pincode"}). Multiple saved addresses are Post-V1.

Database Work

Review Constraints:

```
sql
ALTER TABLE reviews ADD CONSTRAINT unique_product_customer_review UNIQUE (product_id, customer_id);
-- One review per product per customer enforced at DB level
-- Purchase validation (delivered order) enforced at application level
```

Backend Work (Customer & Review Endpoints)

Endpoint	Method	Purpose
/api/customer/profile	GET/PUT	Profile management (name, phone, address with State, consent withdrawal)
/api/customer/wishlist	GET	Get wishlist
/api/customer/wishlist	POST	Add to wishlist
/api/customer/wishlist/:productId	DELETE	Remove from wishlist
/api/reviews	POST	Submit review (registered only)
/api/reviews/:id	PUT	Edit review
/api/reviews/check-eligibility	POST	Check if customer can review product
/api/admin/reviews	GET	List reviews with filters
/api/admin/reviews/:id/moderate	PATCH	Approve/reject/hide/delete

Review Submission Validation:

```
typescript
async function submitReview(req, res) {
  const { productId, orderId, rating, title, comment, photo } = req.body;
  const customerId = req.user.id;

  // 1. Verify delivered order containing this product
  const hasDeliveredOrder = await db.orders.findFirst({
    where: {
```

```

        customer_id: customerId,

        status: 'delivered',

        order_items: { some: { variant: { product_id: productId } } }
    }
});

if (!hasDeliveredOrder) {
    return res.status(403).json({ error: 'You can only review products you have purchased and received' });
}

// 2. Check if already reviewed

const existingReview = await db.reviews.findFirst({
    where: { product_id: productId, customer_id: customerId }
});

if (existingReview) {
    return res.status(409).json({ error: 'You have already reviewed this product' });
}

// 3. Create review with status='pending'

const review = await db.reviews.create({
    data: {
        product_id: productId, customer_id: customerId,
        order_id: hasDeliveredOrder.id,
        rating, title, comment, photo_url: photo,
        status: 'pending'
    }
});

// No email notification is sent to the customer or admin.

return res.json({ success: true, review });
}

```

Review Moderation State Machine:

typescript

```

const REVIEW_STATUS_TRANSITIONS = {
    pending: { canTransitionTo: ['approved', 'rejected'] },
    approved: { canTransitionTo: ['hidden'] },
    rejected: { canTransitionTo: [] },
    hidden: { canTransitionTo: ['approved'] }
};

```

Frontend Work (Customer)

Screen	Features
/customer/dashboard	Overview: recent orders, wishlist preview, review pending count
/customer/profile	Edit name, phone, address (with State field), consent withdrawal
/customer/wishlist	Grid of wishlist products, remove buttons, add to cart
/customer/reviews	List submitted reviews with status badges
/product/[slug]/review	Review form (only if eligible)

Testing Requirements:

- Only registered customers see "Write a Review" button.
- Guest clicking review link redirects to login.
- Customer without delivered order cannot submit.
- Customer with delivered order can submit (once per product).
- Review shows as "pending" after submission (no email sent).
- Admin approves → review appears on product page.
- Admin rejects → review never appears.
- Admin hides → previously public review disappears.
- Customer edits review → status returns to pending.
- Profile edit includes State field in address.
- No password_hash field anywhere in customer responses.

Deliverables:

- Complete customer dashboard.
- Review system fully functional.
- Wishlist working.

Phase 7: Content Management & CMS

Objective: Homepage editor (including Customer Reviews configuration), FAQ management, global settings, SEO configuration.

Database Work

Homepage Content Singleton:

sql

-- Ensure exactly one row exists (includes customer_reviews column - Resolution #6)

INSERT INTO homepage_content (

id, hero_banner, featured_product_ids, categories_section,

why_choose_us, customer_reviews, footer

```
) VALUES (
    1,
    '{"image_url": "/placeholder-hero.jpg", "text": "Welcome to Indicore Originals", "cta_text": "Shop Now", "cta_link": "/shop"}',
    '{}',
    '{"display_categories": []}',
    '[{"icon": "truck", "title": "Free Shipping", "description": "On orders above ₹299"}]',
    '{"max_reviews_to_show": 6}',
    '{"contact_text": "support@indicoreoriginals.com", "copyright_text": "© 2024 Indicore Originals"}'
) ON CONFLICT (id) DO NOTHING;
```

Settings Key-Value Store:

```
sql
INSERT INTO settings (key, value) VALUES
    ('cod_min_order_value', '{"value": 299}'),
    ('cod_max_order_value', '{"value": 50000}'),
    ('store_name', '{"value": "Indicore Originals"}'),
    ('store_email', '{"value": "support@indicoreoriginals.com"}'),
    ('store_phone', '{"value": "+91XXXXXXXXXX"}'),
    ('social_links', '{"instagram": "https://instagram.com/indicore", "facebook": "https://facebook.com/indicore"}');
```

-- Note: There is no email_smtp_config or similar in settings. V1 does not send any automated emails.

Backend Work (CMS Endpoints)

Endpoint	Method	Purpose
/api/homepage	GET	Public homepage content
/api/admin/homepage	GET/PUT	Admin get/update content
/api/faq	GET	Public FAQ list
/api/admin/faq	CRUD	FAQ management
/api/settings	GET	Public settings (COD min/max, store info)
/api/admin/settings	GET/PUT	Admin settings management

Frontend Work - Admin Homepage Editor

Section	Editable Fields
Hero Banner	Image upload, text, CTA text, CTA link

Section	Editable Fields
Featured Products	Product selector (multi-select from product list)
Categories Section	Select which categories to display
Why Choose Us	Array: {icon, title, description} (add/remove/reorder)
Customer Reviews	Max reviews to show (integer input) — stored in <code>homepage_content.customer_reviews.max_reviews_to_show</code>
FAQ Section	Auto-populated from FAQ table (read-only in homepage editor; manage via FAQ Manager)
Footer	Contact text, social links, copyright text

Customer Reviews Settings (Admin):

tsx

// In the Homepage Editor - Customer Reviews section

```
<div>
  <label>Maximum reviews to display on homepage</label>
  <input
    type="number"
    min={1}
    max={20}
    value={homepageContent.customer_reviews.max_reviews_to_show}
    onChange={e => updateSection('customer_reviews', { max_reviews_to_show: parseInt(e.target.value) }})
  />
</div>
```

Storefront - Homepage Customer Reviews Section:

tsx

// Fetch max_reviews_to_show from homepage_content

```
const maxReviews = homepageContent.customer_reviews?.max_reviews_to_show ?? 6;
```

// Query: approved reviews ordered by date, limited

```
const reviews = await db.reviews.findMany({
  where: { status: 'approved' },
  orderBy: { reviewed_at: 'desc' },
  take: maxReviews,
  include: { customer: { select: { name: true } }, product: { select: { name: true } } }
});
```

Testing Requirements:

- Homepage editor updates reflect immediately on storefront.
- Customer Reviews section: changing max count updates displayed count on homepage.
- Featured products selector shows only published products.
- FAQ reorder works (drag-and-drop).
- COD min/max changes affect checkout validation.
- Cannot add/remove homepage sections (fixed 7-section structure).

Deliverables:

- Fully functional homepage editor with Customer Reviews configuration.
- FAQ management working.
- Global settings configurable (no email settings).

Phase 8: Analytics Dashboard

Objective: Admin analytics dashboard with date range filtering.

Backend Work

typescript

// GET /api/admin/analytics?range=today&start=2024-01-01&end=2024-01-31

```
async function getAnalytics(req, res) {
```

```
  const { startDate, endDate } = parseDateRange(req.query);
```

```
  const totalOrders = await db.orders.count({ where: { created_at: { gte: startDate, lte: endDate } } });
```

```
  const revenue = await db.orders.aggregate({
```

```
    where: { created_at: { gte: startDate, lte: endDate }, status: { not: 'cancelled' } },
```

```
    sum: { cart_total: true }
  });
```

```
  const customerCount = await db.customers.count({ where: { created_at: { gte: startDate, lte: endDate } } });
```

```
  const subscriberCount = await db.customers.count({ where: { email_consent_given: true } });
```

```
  const aov = totalOrders > 0 ? (revenue._sum.cart_total ?? 0) / totalOrders : 0;
```

```
  return res.json({
```

```
    business: { totalOrders, revenue: revenue._sum.cart_total, customerCount, aov, subscriberCount },
```

```
    cod: await getCodMetrics(startDate, endDate),
```

```
    reviews: await getReviewMetrics(startDate, endDate),
```

```
    products: await getTopProducts(startDate, endDate)
  });
```

```
}
```

Frontend Work

Component	Features
Date Range Picker	Today, Last 7 Days, Last 30 Days, Custom Range
KPI Cards	Total Orders, Revenue, Customers, Subscribers, AOV
COD Metrics	Verification Rate, Cancellation Rate, Return Rate
Review Metrics	Reviews Submitted, Approval Rate, Avg Rating, Pending Count
Top Products Table	Product name, views, add-to-carts, orders

Testing Requirements:

- Date range filter updates all metrics correctly.
- Revenue excludes cancelled orders.
- Subscriber count only counts email_consent_given = true.
- COD metrics calculate correctly.

Deliverable: Analytics dashboard with date filtering. All metrics from PRD displayed.

Phase 9: Performance & Security Hardening

Objective: Optimize performance, implement security measures, DPDP Act compliance audit.

Performance Optimization

Area	Action	Target
Database	Add compound and partial indexes	Query < 200ms
Images	next/image with priority for LCP	LCP < 2.5s
Code splitting	Dynamic imports for admin pages	Initial bundle < 200KB
ISR	Product listing revalidate=3600, detail revalidate=60	Freshness vs speed
CDN	Vercel edge network	Global < 100ms TTFB

Indexes:

```
sql
```

```
-- Compound indexes for common queries
```

```
CREATE INDEX CONCURRENTLY idx_orders_customer_id_status ON orders(customer_id, status);
```

```
CREATE INDEX CONCURRENTLY idx_orders_created_at ON orders(created_at);
```

```
CREATE INDEX CONCURRENTLY idx_reviews_product_id_status ON reviews(product_id, status);
```

```
CREATE INDEX CONCURRENTLY idx_order_items_order_id ON order_items(order_id);
CREATE INDEX CONCURRENTLY idx_order_items_variant_id ON order_items(variant_id);
-- Partial indexes for filtered queries
CREATE INDEX idx_orders_pending ON orders(status, created_at) WHERE status = 'pending';
CREATE INDEX idx_reviews_pending ON reviews(status, created_at) WHERE status = 'pending';
```

Security Hardening

Area	Requirement	Implementation
Headers	CSP, X-Frame-Options, X-Content-Type-Options	next.config.js
Input sanitization	No HTML in user inputs	Zod + React escaping
CSRF	SameSite cookies	SameSite=Lax cookies
Rate limiting (login)	5 attempts / 15 min	customers table (DB-based, no Redis)
Rate limiting (tracking)	10 attempts / 15 min	order_track_attempts table (DB-based, no Redis)
Admin session	Manual logout	No idle timeout — admin logs out via logout button
Customer session	7 days	Supabase JWT expiry
Password policy	Admin: 16+ chars	Enforce in seed script

Security Headers (next.config.js):

```
javascript
const securityHeaders = [
  { key: 'Strict-Transport-Security', value: 'max-age=63072000; includeSubDomains' },
  { key: 'X-XSS-Protection', value: '1; mode=block' },
  { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
  { key: 'X-Content-Type-Options', value: 'nosniff' },
  { key: 'Referrer-Policy', value: 'origin-when-cross-origin' },
  { key: 'Content-Security-Policy', value: "default-src 'self'; img-src 'self' data: https://*.supabase.co; script-src 'self' " },
];
```

DPDP Act Compliance Audit

- Consent checkbox NOT pre-ticked on checkout.
- Email field labeled "For order updates" (not "Subscribe").
- Consent is optional - order placement proceeds without it.
- Consent withdrawal available in customer profile settings.
- Data export endpoint (/api/customer/export-data) working.

- Account deletion endpoint (/api/customer/delete-account) working.
- Privacy policy page accessible.
- Terms & conditions page accessible.
- Return policy page accessible.

Deliverables:

- Lighthouse score > 90 on mobile.
- Security headers implemented.
- DPDP compliance verified.
- Performance monitoring set up.

Phase 10: Production Deployment

Objective: Deploy to production, verify all features, monitor launch.

Pre-Launch Checklist

Environment:

- Staging environment mirrors production.
- Production Supabase project created (Mumbai region).
- Production Vercel project connected.
- Environment variables set (no secrets in code).
- Custom domain configured (SSL auto-renewing).

Database:

- Migrations run on production.
- RLS policies enabled on all tables (including INSERT policies for guests).
- Initial admin account created via seed script.
- homepage_content singleton row seeded (including customer_reviews column).
- Default settings seeded.
- Backup scheduled (Supabase Pro includes daily backups).
- order_track_attempts cleanup scheduled (pg_cron or Vercel cron).

Testing:

- Full regression test on staging.
- Load test (100 concurrent users).
- Security scan (no vulnerabilities).
- Mobile responsiveness on real devices (iPhone, Android).
- Rate limiting tested (login DB-based + tracking DB-based).
- Multi-tag filter tested.
- Cart clearing after checkout tested.

- State field in all address forms tested.

Monitoring:

- Vercel Analytics enabled.
- Error tracking (Sentry optional).
- Uptime monitoring (UptimeRobot free).
- Logging configured (Vercel logs).

Launch Day Steps

1. Freeze development (no merges 24 hours before).
2. Final staging verification (smoke test all user flows).
3. Production deployment (Vercel from main branch).
4. Run migrations on production database.
5. Verify admin access (login with seed admin).
6. Smoke test:
 - Browse products.
 - Add to cart (multi-tag filter working).
 - Checkout as guest (with State field).
 - Confirm consent is optional.
 - Track order.
7. Monitor error rates (first hour).
8. Announce launch.

Post-Launch (Week 1)

- Daily error log review.
- Monitor database performance.
- Admin manually checks orders dashboard regularly (no automated alerts).
- Fix critical bugs within 24 hours.
- Plan Post-V1 features: email notifications, multiple saved addresses.

4. Development Order (Critical Path)

Week	Focus	Key Deliverables	Dependencies
1	Database + Auth	Schema (14 tables, customer_reviews in homepage_content), RLS (incl. INSERT policies), login	None
2	Admin Product Mgmt	Product CRUD, variants, images, multi-tag	Auth
3	Storefront	Homepage (7 sections), product listing with multi-tag filter, detail, cart	Product API

Week	Focus	Key Deliverables	Dependencies
4	Checkout + Orders	Checkout form (with State), optional consent, cart clear, order creation, inventory	Cart, Product
5	Order Tracking + Admin Orders	Guest tracking, admin order management (pending count), manual COD workflow	Orders
6	Customer Features	Registration, dashboard, single address (with State), wishlist, account linking	Orders
7	Reviews (Customer)	Review submission, eligibility check, display	Orders
8	Reviews (Admin)	Moderation dashboard	Reviews
9	CMS + Settings	Homepage editor (with Customer Reviews config), FAQ, settings	—
10	Analytics + Launch	Analytics, performance, security, deployment	Orders

5. Database Implementation Checklist

Tables to Create (14 tables)

Table	Primary Key	Key Constraints	Notes
customers	UUID (references auth.users)	email UNIQUE, phone NOT NULL	No password_hash column — Supabase Auth owns passwords
categories	bigserial	name UNIQUE, slug UNIQUE	—
tags	bigserial	name UNIQUE	—
products	bigserial	slug UNIQUE, category_id FK	—
product_variants	bigserial	sku UNIQUE, product_id FK, (product_id, variant_type, variant_value) UNIQUE	is_out_of_stock maintained by trigger
product_tags	composite	product_id FK, tag_id FK	—
orders	bigserial	order_number UNIQUE, customer_id FK	shipping_address stored as JSON with state field
order_items	bigserial	order_id FK, variant_id FK	—

Table	Primary Key	Key Constraints	Notes
reviews	bigserial	(product_id, customer_id) UNIQUE	—
wishlist	composite	customer_id FK, product_id FK	—
faq	bigserial	—	display_order for reordering
homepage_content	bigserial	Singleton (only 1 row)	Includes customer_reviews jsonb column
settings	text (key)	key PK	No email config keys
order_track_attempts	bigserial	ip_address, expires_at	Cleaned up via cron

customers Table - Column Reference

Column	Type	Description
id	UUID (FK to auth.users)	Primary key
email	TEXT UNIQUE	Customer email
name	TEXT	Full name
phone	TEXT NOT NULL	Phone number (non-unique)
address	JSONB	Single address: {"street","city","state","pincode"}
role	TEXT	'admin' or 'customer'
is_registered	BOOLEAN	false for guest checkouts
email_consent_given	BOOLEAN	Marketing opt-in
login_attempts	INTEGER	For rate limiting
locked_until	TIMESTAMP	For rate limiting
created_at	TIMESTAMP	—
updated_at	TIMESTAMP	—

Column	Type	Description
<i>(No password_hash)</i>	—	—

homepage_content Table - Column Reference

sql

```
CREATE TABLE homepage_content (
  id bigserial PRIMARY KEY,
  hero_banner jsonb, -- {"image_url","text","cta_text","cta_link"}
  featured_product_ids bigint[],
  categories_section jsonb, -- {"display_categories": [id,...]}
  why_choose_us jsonb, -- [{"icon","title","description"},...]
  customer_reviews jsonb DEFAULT '{"max_reviews_to_show": 6}',
  -- FAQ section auto-populates from faq table; no config needed here
  footer jsonb, -- {"contact_text","social_links","copyright_text"}
  updated_at timestamp DEFAULT now()
);
```

Indexes (Summary)

- idx_customers_phone, idx_customers_role
- idx_products_category_id, idx_products_price
- idx_product_variants_product_id, idx_product_variants_stock
- idx_product_tags_tag_id
- idx_orders_customer_id, idx_orders_status, idx_orders_created_at, idx_orders_customer_status
- idx_order_items_order_id, idx_order_items_variant_id
- idx_reviews_product_id, idx_reviews_customer_id, idx_reviews_status, idx_reviews_product_status
- idx_order_track_ip, idx_order_track_expires

Triggers & Functions

1. is_out_of_stock **trigger** (modify NEW only - do NOT update same table)
2. updated_at **trigger** (all tables with updated_at)
3. **Order number generation function** (generate_order_number)

Seed Data (Initial Admin, Settings, Homepage)

- Admin account (run after auth.users created). No password_hash in customers table - password is in auth.users only.
- Default settings (no email config).
- Homepage content singleton (includes customer_reviews).

6. Backend Implementation Checklist

API Endpoints (35 total)

Auth (4 endpoints):

- /api/auth/login (POST, Public, Rate limit: 5/15min per email - DB-based)
- /api/auth/signup (POST, Public)
- /api/auth/logout (POST, Optional)
- /api/auth/session (GET, Optional)

Public Storefront (7 endpoints):

- /api/products (GET, Public)
- /api/products/:slug (GET, Public)
- /api/categories (GET, Public)
- /api/tags (GET, Public)
- /api/search (GET, Public)
- /api/homepage (GET, Public)
- /api/faq (GET, Public)
- /api/settings (GET, Public)

Checkout & Orders (4 endpoints):

- /api/orders (POST, Public)
- /api/orders/validate (POST, Public)
- /api/orders/track (GET, Public, Rate limit: 10/15min per IP - DB-based)
- /api/orders/guest-history (GET, Public)

Customer (Registered) (6 endpoints):

- /api/customer/profile (GET/PUT, Customer)
- /api/customer/orders (GET, Customer)
- /api/customer/wishlist (GET/POST, Customer)
- /api/customer/wishlist/:id (DELETE, Customer)
- /api/customer/export-data (GET, Customer)
- /api/customer/delete-account (DELETE, Customer)

Reviews (3 endpoints):

- /api/reviews (POST, Customer, Rate limit: 5/hour)
- /api/reviews/:id (PUT, Customer)
- /api/reviews/check-eligibility (POST, Customer)

Admin (11 endpoints):

- /api/admin/products (CRUD, Admin)
- /api/admin/categories (CRUD, Admin)
- /api/admin/tags (CRUD, Admin)
- /api/admin/orders (GET/PATCH, Admin)
- /api/admin/reviews (GET/PATCH, Admin)

- /api/admin/homepage (GET/PUT, Admin)
- /api/admin/faq (CRUD, Admin)
- /api/admin/settings (GET/PUT, Admin)
- /api/admin/analytics (GET, Admin)
- /api/admin/upload/image (POST, Admin)

No /api/email or email service endpoints exist in V1. No Redis/Upstash endpoints or configuration exist anywhere.

Validation Schemas (Zod)

- EmailSchema, PhoneSchema, RatingSchema
- AddressSchema (**includes State** - Resolution #8)
- CheckoutSchema (**consent is OPTIONAL** - Resolution #7)
- ProductSchema (admin)
- ReviewSchema

Multi-Tag Filtering (Resolution #9)

typescript

```
// GET /api/products?tags=3,7,12&category=shirts&inStock=true
```

```
const tagParam = searchParams.get('tags');
```

```
const tagIds: number[] = tagParam
```

```
  ? tagParam.split(',').map(Number).filter(n => !isNaN(n) && n > 0)
```

```
  : [];
```

```
// Products must have at least one of the selected tags
```

Inventory Logic (Transaction with Row-Level Locks)

- Lock variants (FOR UPDATE).
- Verify stock.
- Deduct stock.
- Create order (address stored as JSON with state).
- Create order items.
- **No email notification.** Admin will see this order in /admin/orders dashboard.

7. Frontend Implementation Checklist

Customer Screens (27 screens)

Screen	Route	Priority
Homepage	/	P0
Product Listing	/shop	P0
Product Detail	/product/[slug]	P0
Category Page	/category/[slug]	P1
Search Results	/search?q=	P1
Cart	/cart	P0
Checkout (with State)	/checkout	P0
Order Confirmation	/order/confirmation/[id]	P0
Track Order	/track-order	P0
Guest Order History	/guest-orders	P0
Login	/login	P0
Signup	/signup	P0
Customer Dashboard	/customer/dashboard	P1
Customer Orders	/customer/orders	P1
Customer Order Detail	/customer/orders/[id]	P1
Customer Profile (single address with State)	/customer/profile	P1
Wishlist	/customer/wishlist	P1
My Reviews	/customer/reviews	P2
Submit Review	/product/[slug]/review	P2
Edit Review	/review/[id]/edit	P2

Screen	Route	Priority
Dashboard (pending order count)	/admin/dashboard	P0
Screen	Route	Priority
Products	/admin/products	P0
FAQ	/faq	P1
Add Product	/admin/products/new	P0
Contact	/contact	P2
Edit Product	/admin/products/[id]/edit	P0
About Us	/about	P2
Categories	/admin/categories	P1
Privacy Policy	/privacy	P1
Tags	/admin/tags	P1
Terms & Conditions	/terms	P1
Orders (pending badge)	/admin/orders	P0
Return Policy	/returns	P1
Order Detail (address with State)	/admin/orders/[id]	P0
404	/404	P1
Reviews	/admin/reviews	P1
Customers	/admin/customers	P2
Homepage Editor (incl. Customer Reviews config)	/admin/homepage	P1
FAQ Manager	/admin/faq	P1
Analytics	/admin/analytics	P1
Settings	/admin/settings	P1
Admin Profile	/admin/profile	P2
Login	/login	P0

Admin Screens (16 screens)

Form Validation (Client-side)

Checkout validation - includes State, consent optional (Resolutions #7, #8):

typescript

```
const checkoutSchema = z.object({
  fullName: z.string().min(2, 'Name must be at least 2 characters'),
  phone: z.string().regex(/^[0-9]{10}$/, 'Enter 10-digit phone number'),
  street: z.string().min(5, 'Enter complete street address'),
  city: z.string().min(2, 'Enter city'),
  state: z.string().min(2, 'Enter state'), // Resolution #8
  pincode: z.string().regex(/^[0-9]{6}$/, 'Enter 6-digit pincode'),
```

```
email: z.string().email('Enter valid email'),
consent: z.boolean().default(false) // Resolution #7: optional
});
```

Cart Clearing After Order (Resolution #10)

- Clear localStorage cart **immediately** after successful order response, **before** redirect.

Multi-Tag Filter UI (Resolution #9)

- Use multi-select checkboxes in filter sidebar.
- Sync to URL: ?tags=id1,id2,id3 (comma-separated).

Loading & Error States

- **Loading:** Skeleton loaders for product grid (6 cards), spinner for add to cart button.
- **Error:** Network error, form error, stock error, COD range error messages.

8. Security Implementation Plan

Authentication Security

Requirement	Implementation
Passwords hashed	Supabase Auth (bcrypt, 10 rounds in auth.users)
No password_hash in customers table	Supabase Auth is sole owner of passwords
JWT storage	HTTP-only cookie, Secure, SameSite=Lax
Customer session	7 days (Supabase JWT expiry)
Admin session	Manual logout only — no idle timeout
Password policy	Admin: 16+ chars, unique; enforced in seed script
Initial admin	Seed script only; no public admin registration route

Rate Limiting (Database-based - No Redis, No Upstash)

Endpoint	Limit	Window	Storage	Notes
Login	5 attempts	15 min	customers.login_attempts / locked_until	Per email
Order tracking	10 attempts	15 min	order_track_attempts	Per IP; rolling window
Review submission	5 submissions	1 hour	Application logic	Per customer

There is no Redis, Upstash, or any external caching service in V1. All rate limiting uses Supabase PostgreSQL.

Row Level Security (RLS)

- Principle: Least privilege - every table has RLS enabled.
- Service role bypasses RLS for admin server-side operations only.

Critical: Guest INSERT policies - The following tables must have INSERT policies that allow unauthenticated users for guest checkout to work:

- customers - allow INSERT for guest creation
- orders - allow INSERT for guest order creation
- order_items - allow INSERT for order item creation
- order_track_attempts - allow INSERT for rate limiting

Without these INSERT policies, guest checkout will fail with RLS violations.

Admin Route Protection (Middleware)

- Checks role in customers table on every /admin/* request.
- Redirects to /login if not authenticated or not admin.

9. Testing Strategy

Unit Testing

- generateOrderNumber: unique, format correct.
- calculateCartTotal: multiplies quantity × price.

End-to-End Testing (Playwright)

- Guest checkout with State field.
- Multi-tag filter selects multiple tags.
- Consent is optional.

Security Testing

- Attempt to access /admin without login → redirect to /login.
- Attempt to access /api/admin/* without admin JWT → 401.
- Attempt SQL injection → fails Zod validation.
- Attempt XSS in review comment → React escapes output.
- Attempt to view another user's order → 403.
- Login rate limiting: 5 failures → 15-minute logout.
- Order tracking: 10 requests → 429 (no Redis → DB table based).
- Guest checkout with no auth → order created (RLS INSERT policy works).
- Attempt to access customer with password_hash field → field not returned (doesn't exist).

10. Deployment Plan

Environment Setup

Environment	URL	Database	Branch
Development	localhost:3000	Local Supabase (Docker)	feature/*
Staging	https://staging.indicoreoriginals.com	indicore-staging	staging
Production	https://indicoreoriginals.com	indicore-prod	main

Supabase Production Setup

1. Create Supabase project (Mumbai region).
2. Run migrations.
3. Auth settings: enable email/password only; disable phone/SMS provider.
4. Create storage buckets: product-images, review-photos (both public).
5. Seed database (admin account + settings + homepage_content).
6. Schedule order_track_attempts cleanup (if pg_cron available, otherwise Vercel cron job).

Environment Variables

Variable	Secret
NEXT_PUBLIC_SUPABASE_URL	https://gefenowenqvlhbrfnibl.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImdlZmVub3dlbnF2bGhicmZuaWJsIiwicm9sZSI6ImFub24iLCJpYXQiOiJlZ3ODE1MDA0NjcsImV4cCI6IjA5NzA3NjQ2N30uZD1veJPNwjdDz5HiTF7WkItbAw_AFxwdrqQWdvQFFEJw
SUPABASE_SERVICE_ROLE_KEY	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImdlZmVub3dlbnF2bGhicmZuaWJsIiwicm9sZSI6ImNlcnpY2Vfc3ZSImlhdCI6MTc4MTUwMDQ2NywiZXhwIjoyMDk3MDc2NDY3fQ.6n7sE7OrwFCPrqt_Di7Bnd23qikYlkgGf_A8Wy1yWDQ
NEXT_PUBLIC_APP_URL	No
IMAGE_OPTIMIZATION_SECRET	99971ddf324acd66b141be24059fc0601bd6ea1b90a7bab39a858c65c37248ca

No SMTP_HOST, EMAIL_API_KEY, RESEND_API_KEY, or similar - V1 sends no automated emails.

No UPSTASH_REDIS_URL or REDIS_TOKEN - V1 uses database-based rate limiting only.

Rollback Strategy

- **Application rollback (Vercel):** vercel rollback <deployment_id>
- **Database rollback:** supabase migration repair --status reverted <migration_version> OR restore from Supabase Pro daily backup.

11. Cost Breakdown

MVP Phase (0-3 months) - ~\$1/month

Service	Plan	Cost
Vercel	Hobby	\$0
Supabase	Free tier	\$0
Domain.com (annual)	—	~\$1/month
Total		~\$1/month

Growth Stage (3-9 months) - ~\$46/month

Service	Plan	Cost
Vercel	Pro	\$20
Supabase	Pro (includes pg_cron for cleanup)	\$25
Domain	—	\$1
Total		\$46/month

12. Production Readiness Checklist

Functionality (52 items)

Authentication & Accounts:

- Single login page works for admin and customers.
- Admin login redirects to /admin/dashboard.
- Customer login redirects to /customer/dashboard.
- 5 failed login attempts lock account for 15 minutes (database-based, no Redis).
- Guest can checkout without account.
- Guest can register after checkout.
- Guest orders link to account when registering with same email.
- Admin session: manual logout only (no idle timeout).
- Customer session persists for 7 days.

Products & Browsing:

- Admin can create product with up to 10 gallery images.
- Product variants (size, color, material, custom) work.
- Each variant has unique SKU and stock quantity.
- Out-of-stock variants disabled on product page.

- Product slug auto-generates from name with collision handling.
- SEO fields per product (title, meta description).
- Multi-tag filtering works (tags=id1,id2 returns products with any of the selected tags).
- Search returns relevant results.
- Product listing filters (category, tags, price, in-stock).

Cart & Checkout:

- Cart persists after page refresh (localStorage).
- Cart is cleared from localStorage immediately after successful order placement.
- COD min/max values enforced (Place Order disabled when out of range).
- Checkout form collects: name, phone, street, city, state, pincode, email.
- State field is present and required in checkout form.
- Consent checkbox NOT pre-ticked.
- Order placement succeeds when consent is unchecked (consent is optional).
- Order placement succeeds when consent is checked.
- No email or notification triggered after order placement.
- Inventory deducted immediately on order placement.
- Order status = 'pending' after placement.

Order Management:

- Admin can view orders with status filters.
- Admin orders list shows pending count prominently (manual monitoring, no automated alerts).
- Admin can manually mark order as Confirmed (COD verification).
- Order status updates: pending → confirmed → shipped → delivered.
- Guest can track order with Order ID + Email (address shown includes State).
- Rate limiting: 10 tracking attempts per IP per 15 minutes (database-based).
- Guest can view all orders by entering email.
- Return request workflow works.
- No email sent at any order status transition.

Reviews:

- Only registered customers see "Write a Review" button.
- Only customers with delivered orders can review.
- One review per product per customer.
- Review submission requires rating, title, comment.
- Optional photo upload (max 5MB).
- Pending reviews not public.

- Admin can approve/reject/hide/delete reviews.
- Approved reviews appear on product page.
- Customer can edit review (returns to pending).

CMS:

- Homepage has exactly 7 sections (no more, no less).
- Admin can edit all homepage sections via editor.
- Admin can configure max_reviews_to_show in Customer Reviews section.
- Homepage displays correct number of reviews per customer_reviews.max_reviews_to_show.
- FAQ page and homepage section auto-populate from FAQ table.
- Admin can reorder FAQs via drag-and-drop.
- Global settings (COD min/max, store info) updatable.

Analytics:

- Date range filter: Today, Last 7 Days, Last 30 Days, Custom.
- All metrics update with date range.
- Subscriber count reflects email_consent_given = true only.

Security (18 items)

- Passwords hashed with bcrypt (Supabase Auth in auth.users).
- No password_hash column in customers table.
- JWT stored in secure cookie (SameSite=Lax).
- Admin routes protected by middleware (role check on every request).
- No admin idle session timeout - admin logs out manually.
- All API endpoints have authentication/authorization.
- RLS policies enabled on all 14 tables.
- Guest INSERT policies exist for: customers, orders, order_items, order_track_attempts.
- RLS policies tested (customers can't see other customers' data).
- Rate limiting on login: 5 attempts/15min (DB-based, no Redis).
- Rate limiting on order tracking: 10 attempts/15min (DB-based, no Redis).
- Input validation with Zod on all endpoints.
- Security headers implemented (CSP, X-Frame-Options, etc.).
- Admin password: 16+ characters, in password manager.
- Supabase service role key stored as Vercel secret.
- No API keys in client code.
- File uploads validated (type, size).
- order_track_attempts cleanup scheduled.

Performance (10 items)

- Lighthouse mobile score > 90.
- LCP < 2.5s on 3G.
- FCP < 1.5s.
- CLS < 0.1.
- Product listing API responds < 300ms (p95).
- Images optimized (WebP, max width 1200px).
- next/image with lazy loading.
- Database indexes created.
- ISR implemented for product listing (revalidate = 3600).
- Compound and partial indexes added for filtered queries.

Compliance (8 items)

- Consent checkbox not pre-ticked.
- Order placement works with consent=false.
- Email field labeled "For order updates".
- Consent withdrawal available in customer profile.
- Data export endpoint working.
- Account deletion endpoint working.
- Privacy policy, T&C, and Return policy pages accessible.

Testing (17 items)

- Unit tests pass (80% coverage).
- Integration tests pass (all API endpoints).
- E2E tests pass (critical user flows).
- State field tested in checkout, order detail, and profile.
- Multi-tag filter tested (tags=id1,id2).
- Cart cleared after order placement tested.
- Consent optional tested (order succeeds without consent).
- Customer Reviews config tested (homepage reflects max_reviews_to_show).
- OWASP ZAP scan (no high vulnerabilities).
- Load test (100 concurrent users).
- Mobile tested on iPhone 12, Pixel 5.
- Cross-browser: Chrome, Safari, Firefox.
- Rate limiting tested (DB-based; no Redis dependency).
- Inventory race condition tested.

- RLS INSERT policies tested for guest checkout.
- No email triggered at any point tested.
- Rollback procedure tested.

Deployment (10 items)

- Staging environment mirrors production.
- Environment variables set correctly (no Redis/email env vars).
- Database migrations tested on staging.
- Seed script creates admin, settings, and homepage_content singleton.
- Backup strategy configured (Supabase Pro).
- Monitoring setup (Vercel Analytics).
- Error tracking (Sentry optional).
- Uptime monitoring enabled.
- Rollback procedure documented and tested.
- Custom domain with SSL.

Appendix A: Shipping Address JSON Standard

All shipping addresses throughout the system are stored and transmitted as structured JSON. The following format is the single source of truth.

json

```
{  
  "street": "123 Main Street, Sector 5",  
  "city": "Gurgaon",  
  "state": "Haryana",  
  "pincode": "122001"  
}
```

This applies to:

- POST /api/orders request body.
- orders.shipping_address database column (stored as JSON string or jsonb).
- Order detail responses (admin and customer views).
- Guest order tracking response.
- Customer profile address field (customers.address).
- All Zod address validation schemas.

Appendix B: Resolution Reference (Quick Lookup)

Resolution	Where Applied
#1 No email notifications	§1.4, §1.5, Phase 4, Phase 5, Phase 6, §6 endpoints, §12 checklist
#2 DB-based rate limiting	§1.5, Phase 1, Phase 5, §5 tables, §6 rate limiting, §8 security, §12 security
#3 Single address (no multiple)	§1.4 Out of Scope, Phase 6, §5 customers table
#4 No password_hash	Phase 1 DB, §5 customers table, §6 auth flows, §8 security, §12 security
#5 No admin idle timeout	Phase 1 testing, §8 security, §12 security
#6 customer_reviews in homepage_content	Phase 1 DB, Phase 7, §5 homepage_content, §6 seed data, §12 CMS
#7 Consent optional	Phase 4 checkout, §6 schemas, §7 frontend, §9 testing, §12 checklist
#8 State field	Appendix A, Phase 4, Phase 6, §5 tables, §6 schemas & payloads, §7 forms, §9 tests
#9 Multi-tag filter	Phase 3, §6 API, §7 frontend, §9 testing, §12 checklist
#10 Cart clearing	Phase 4, §7 frontend, §9 testing
#11 No admin notifications	§1.4 Out of Scope, Phase 5, §12 order management

Appendix C: Useful SQL Queries

sql

-- Check for negative stock (should never happen)

```
SELECT * FROM product_variants WHERE stock_quantity < 0;
```

-- Find products without variants

```
SELECT p.* FROM products p
LEFT JOIN product_variants pv ON p.id = pv.product_id
WHERE pv.id IS NULL;
```

-- Verify homepage_content singleton (should always return 1 row)

```
SELECT id, jsonb_pretty(customer_reviews) FROM homepage_content;
```

-- Review approval rate

```
SELECT
  COUNT(*) as total,
```

```
SUM(CASE WHEN status = 'approved' THEN 1 ELSE 0 END) as approved,
ROUND(SUM(CASE WHEN status = 'approved' THEN 1 ELSE 0 END) * 100.0 / NULLIF(COUNT(*), 0), 2) as approval_rate
FROM reviews;

-- Daily orders trend (last 30 days)
SELECT DATE(created_at) as date, COUNT(*) as orders
FROM orders
WHERE created_at > NOW() - INTERVAL '30 days'
GROUP BY DATE(created_at)
ORDER BY date DESC;

-- Pending orders count (for admin dashboard badge)
SELECT COUNT(*) as pending_orders FROM orders WHERE status = 'pending';

-- Subscriber count (consented customers)
SELECT COUNT(*) as subscribers FROM customers WHERE email_consent_given = true;

-- Verify no password_hash column exists (should return no rows)
SELECT column_name FROM information_schema.columns
WHERE table_name = 'customers' AND column_name = 'password_hash';

-- Check RLS is enabled on all tables (result should be empty)
SELECT tablename, rowsecurity FROM pg_tables
WHERE schemaname = 'public' AND rowsecurity = false;
```

Appendix D: Troubleshooting Common Issues

Issue	Likely Cause	Solution
Guest checkout returns 403	RLS INSERT policy missing on customers/orders	Add FOR INSERT WITH CHECK (true) policies
Login returns 401 but credentials correct	customers.role not set	Check seed script ran; verify role column
Products not showing on storefront	RLS policy blocking public reads	Verify FOR SELECT USING (true) on products
Variant stock not updating	Trigger not firing	Verify trigger_update_is_out_of_stock exists and is enabled

Issue	Likely Cause	Solution
Order tracking returns 429 too quickly	IP resolving differently	Check order_track_attempts table; verify window is 15 min
Image upload fails	File too large (>5MB)	Add client-side resizing before upload
COD min/max not working at checkout	Settings not seeded	Check settings table has cod_min_order_value and cod_max_order_value
Consent field causing order failures	Consent treated as required	Verify CheckoutSchema has consent: z.boolean().default(false) not .refine()
State missing from stored address	Address stored as string, not JSON	Ensure JSON.stringify(address) includes state field; verify AddressSchema
Multi-tag filter returning all products	tagIds.length === 0 check missing	Verify empty tag array skips filter
Cart not clearing after order	clearCart() called after redirect	Ensure clearCart() is called before router.push()
Homepage reviews count wrong	customer_reviews column not seeded	Run seed or check homepage_content row; add missing column